

Question: Is the double-blind review process really double-blind?

Response: an analysis on how well deep learning techniques can infer the authors of a paper.

Technique: we explore Convolutional Neural Networks trained on various aspects of a paper (content, style features and references) to understand the extent to which we can infer the authors of a paper and what aspects contribute the most.

- **The double-blind review process implies that the submitted papers have to be anonymized, i.e., the authors' names are not explicitly available with the papers, and any direct or indirect indications of who the authors might be (for example, referring to self-citations in the first person) are forbidden.**
- **Reviewers have access only to the papers' content.**
- **The authors in turn do not know who their assigned reviewers are.**
- **We are questioning here the notion of anonymity in the double-blind review.**

- **We are interested in understanding how predictable the authorship of a paper is, and specifically what part of the paper gives it away.**
- **In order to do this:**
 - ✓ **we train deep learning models on papers published in the ACL and EMNLP conferences, using features extracted from each paper's body of text as well as its references;**
 - ✓ **we perform an in-depth analysis of the predictive value of each feature;**
 - ✓ **we show how the number of authors considered for analysis can affect performance.**

Datasets

- For evaluation, we used two datasets of articles from the computational linguistics conferences ACL and EMNLP, published on or before 2014.
 - The ACL dataset contains 4412 articles authored by a total of 6565 unique authors.
 - The EMNLP dataset is comprised of 1027 articles written by 1861 unique authors in total.
- From each dataset, we normalized the author names to consist of the initial of the first name and the full last name and removed the authors with less than three articles, leaving us with 922 authors for the ACL dataset and 262 authors for the EMNLP dataset (which represent our classes).

Proposed model

- **Machine learning experiment:** we formulate the problem as a supervised classification task, where each article is labelled with one or multiple authors, and **a machine learning model** learns to predict the set of correct labels (authors) for each data point (article). The order of the authors is not taken into consideration.
- **As model,** we choose a neural network with several subcomponents corresponding to various types of features.

Features

In order to capture as many of the aspects of a scientific article as possible in our model, we extract and use various features, corresponding to different levels at which characteristics of the author could manifest. We categorize these into three main types of features:

- ✓ Content level**
- ✓ Style level**
- ✓ Citation level**

- **Content level features: word sequences (consisting of 100-word sequences in the article's title, abstract and body).**
- **Style level features: stopwords bag-of-words, part-of-speech sequences.**
- **Citation level features: bag-of-words of cited authors.**

- **The network is designed to learn from each separate feature using dedicated subcomponents.**
- **At the content level: we use convolutional layers to learn from word sequences. We are passing the word sequences through a single convolutional layer with 300 filters, and a kernel size of 9, followed by a max pooling layer. Before going through the convolutions, the word sequences are passed through a word embedding layer of 300 dimensions, initialized with the pre-trained word2vec embeddings available from Google, trained using the skip-gram objective.**

- **At the style level:** a separate convolutional layer is dedicated to learning from part-of-speech sequences. As in the case of word sequences, we consider the order of parts of speech in a text segment to be relevant, assuming certain types of syntactical constructs can be specific to certain authors. Thus, after tagging a text segment with parts of speech, we encode the result as part-of-speech sequences and pass them through a convolutional layer of 50 filters and kernel size 4, followed by a max pooling layer. Stopwords are extracted from each article segment and encoded as bag-of-words, keeping their frequencies, but not their order in the text. We use the stopwords list available from the NLTK package.

- **At citation level:** to extract knowledge from citations, we focus on cited authors, encoding for each article the authors cited in its references section, along with the citation frequencies for each author. The total number of cited authors in each of our datasets is much larger than the number of authors that contribute directly to one of the articles, e.g., over 22000 unique authors are cited in our ACL dataset. This makes the encoding that we use for cited authors to be very high-dimensional, so we pass the extracted feature through an additional lower-dimensional fully connected layer.

- **Final layers of the network:** in the final layers of the network, we collect all of the output from each subcomponent dealing with the various features, and pass them through a dense component consisting of a fully connected layer and a Softmax layer that produces the network's predicted probabilities for each class.

Preprocessing and Feature Extraction

- **We extracted the text from PDFs using Grobid.**
- **Grobid is a machine learning software for extracting information from scholarly documents. Link to Grobid:**
<https://github.com/kermitt2/grobid>
- **Several preprocessing steps were necessary before using the articles' text as features in the model (see the following slides).**

For text-related features we consider the title, abstract and body of the articles. We exclude references from the article's text, by removing them both from the references section and from the citations within the article text. This is done in order to isolate text features from citation features.

Steps to follow with the resulting text:

- **text preprocessing** (lowercasing every word, discarding numbers and punctuation, resolving end-of-line hyphenation)
- **construct a list of vocabulary words** consisting of the most frequent 50,000 words in all texts. (The literature indicates this vocabulary size as optimal). Note that, for EMNLP, which is a smaller dataset, we restrict the minimum word frequency to 5 occurrences, leaving us with a vocabulary of approximately 23,000 words.

- **Considering only the words in each vocabulary (and replacing all other words with an "unknown" token), encode the text as word sequences of 100 words, padding the sequences with zeros if they are shorter.**
 - **Note that the training examples consist of these word segments, rather than full articles.**
- **Discard outliers, ignoring articles consisting of either zero or more than 20,000 words.**
- **Extract the context around citation mentions within the content of articles, by selecting a window of 100 characters around the citation (and excluding the citation itself), then applying the same text preprocessing steps as above only on this window. (This will be used as a separate feature).**

Part-of-speech features

- **Apply a POS tagger (preferably Stanford POS tagger) to the word sequences. The result consists of part-of-speech sequences corresponding to each article segment. Stopwords are encoded as bag-of-words for each article segment.**

Citation level:

- Citations extracted from the "References" section of each article are encoded as bag-of-authors unordered sets of citation frequencies corresponding to each cited author.
- ✓ Recall that author names are normalized to consist of the initial of the first name and the full last name.

Experimental setup

- **The dataset is split in three subsets**: one for actual training, one for validation (used for tuning hyperparameters), and the third for testing performance.
- **Important remark**: ensure that each of the three sets contains at least one article from each author in our labeled set. This also implies that we exclude any authors with less than three articles.
- **Remember that** our datapoints consist of article segments (word sequences of 100 words) rather than full articles. **Therefore**:
- Ensure that all segments extracted from one given scientific article are appointed to the same set, and not split between training, validation and test.

Note that: we must reduce the impact of the label skewness (asimetria etichetarii) on our trained model. Therefore:

- ✓ **we use weighted sampling for generating the training examples, making sure the probability of generating a training example from any class is approximately the same across classes. For this to be possible, a final adjustment had to be made to our training examples. Our datasets, comprising of scientific articles, with each article having been written either by a single author or in co-authorship between several authors, essentially consists of multi-label examples. For training, we transform the training examples from multi-label examples to single label examples, by generating several copies of the same datapoint, each labelled with only one of its authors, whenever a text was written by more than one author.**

Remarks:

- **This allows us to perform weighted sampling, as well as use a simple softmax layer as the final layer in the network, which generates one predicted label for any training example.**
- **At the evaluation stage, we use the original multilabeled examples, to be able to correctly measure the model's performance.**

METRICS

- **Depending on whether we see the list of authors that contributed to an article as a sorted or unsorted list, a machine learning model that can predict this set of authors can be designed either as a multilabel classifier or as a ranking model.**
- **We disregard the order of the authors of an article, assuming it is not always relevant to the quantity of each author's contribution to the article text. We thus represent the authors for an article as an unordered set of labels.**
- **We do, however, consider the order of the predicted classes in the model's output.**

For evaluating our model, we use both the performance metrics that are suited for multi-class classification (where we treat the model's predictions as unordered sets) and metrics suited to ranking problems, which are generally used in information retrieval (where we see the list of model predictions as a sorted list, ranked according to the probabilities in the model's output).

These performance metrics are the following:

- **Accuracy@k** - computed as the number of articles for which at least one true author was in the top k predicted authors. We use $k = 10$ as this number was shown to perform well in other search and retrieval tasks (Spink and Jansen, 2004).

- **Mean Average Precision (MAP)**

$$MAP = \frac{1}{|\mathcal{A}|} \sum_{a \in \mathcal{A}} \frac{1}{r} \sum_{k=1}^r P@k$$

where $\mathcal{A}$ is the set of articles, and precision at rank k $P@k$ is the number of correct authors within the first k predicted authors relative to the number of predictions (k). Here too we use a maximum rank of $r = 10$.

- **Mean Average Recall (MAR)**

$$MAR = \frac{1}{|\mathcal{A}|} \sum_{a \in \mathcal{A}} \frac{1}{r} \sum_{k=1}^r R@k$$

where recall at rank k $R@k$ is defined as the number of correct authors within the first k predicted authors relative to the total number of true authors ($r = 10$ as before).

- **Mean Reciprocal Rank (MRR)**

$$MRR = \frac{1}{|\mathcal{A}|} \sum_{a \in \mathcal{A}} \frac{1}{r_a}$$

where r_a is the rank at which the first correct predicted author was found for article $a \in \mathcal{A}$.

- **The measured performance on the model's output:**

Since our datapoints are article segments rather than full articles, the measured performance on the model's output will be with respect to these segments: for example, an accuracy of 10% denotes that 10% of the segments were correctly classified.

- **We additionally adapt these metrics to be able to also output the performance at article-level.**
- ✓ **We do this by grouping the article segments in our test set according to the article they were extracted from, and for each article, order all probabilities in the network's output for the given article, and consider the top predicted classes across this global ranking as the model's predictions for the target article.**

Experiment Settings

In order to understand the contribution of each feature for predicting the authors of a paper, we perform an ablation study - isolating and using in turn only certain features and combinations of features and deliberately not using others.

First experiment:

- We compare our model's performance on both the entire set of authors in each dataset, and only a subset of the authors - selecting only the top most prolific ones (top-100, top-200, and so on) and ignoring the rest.
- In this experiment, we only consider articles authored by these selected authors, both for training and test.
- ✓ Remark: We expect performance should be higher on this subset where rare authors are discarded.

Second experiment:

- **We group the features according to the level of the text that they represent: content level (word sequences), style level (stopwords and POS sequences) or citation level (cited authors). We run our model using the features in each of the groups in turn, and ignoring the rest.**
- ✓ **Remark: The measured performance in each separate setting should be an indicator of the importance of the specific feature (or feature combination) and of the aspect of the article that it captures.**

Third experiment:

- We experiment separately with our text features, in order to understand which part of the scientific text is more specific to its author, and how much of the text is really useful in predicting the author.
- **We look separately at:**
 - the entire body of the article;
 - only the title and abstract;
 - only at the context of the citations that occur in the text.
- ✓ **Recommendation:** These last experiments, aiming to analyze feature importance, should be performed in the small-scale setting, where only the top 100 authors are considered.

Results on ACL dataset (segment level)

Features	Acc	MAP	MAR	MRR
All features (top100cls)	<u>84.65</u>	9.87	46.22	25.28
All features (top200cls)	70.00	<u>18.07</u>	54.41	48.96
All features (top500cls)	63.77	16.82	43.34	41.60
All features (mid200cls)	62.38	14.02	49.44	40.78
All features (last200cls)	54.18	11.38	43.22	35.21
All features (922cls)	52.41	13.78	31.29	33.52
Content + Ref (100cls)	<u>75.38</u>	<u>16.74</u>	<u>58.42</u>	<u>49.46</u>
Content (100cls)	49.87	8.87	33.23	25.34
Style ft.	25.21	3.22	13.81	9.20
Content (100cls)	<u>49.87</u>	<u>8.87</u>	<u>33.23</u>	<u>25.34</u>
Title+abstr.	16.19	2.71	9.52	6.22
Ref. contexts	35.53	5.07	21.27	13.54

Results on ACL dataset (article level)

Features	Acc	MAP	MAR	MRR
All features (top100cls)	<u>87.88</u>	<u>24.71</u>	<u>78.37</u>	<u>72.33</u>
All features (top200cls)	66.78	16.93	51.10	46.20
All features (top500cls)	60.74	15.36	39.71	38.68
All features (mid200cls)	54.82	11.14	40.38	32.99
All features (last200cls)	50.00	8.61	36.24	25.53
All features (922cls)	47.59	11.74	27.04	28.77
Content + Ref (100cls)	76.13	16.37	57.50	48.73
Content (100cls)	48.29	10.51	37.16	31.44
Style ft.	26.70	3.18	14.25	9.26
References	<u>86.67</u>	<u>24.35</u>	<u>78.31</u>	<u>71.37</u>
Content (100cls)	<u>48.29</u>	<u>10.51</u>	<u>37.16</u>	<u>31.44</u>
Title+abstr.	18.08	2.56	10.23	6.34
Ref. contexts	30.95	4.15	11.20	18.16

Results on EMNLP dataset (segment level)

Features	Acc	MAP	MAR	MRR
All features (top100cls)	<u>79.97</u>	<u>19.98</u>	<u>60.14</u>	<u>55.33</u>
All features (mid100cls)	57.10	12.02	43.70	35.52
All features (last100cls)	62.96	12.02	46.10	36.08
All features (262cls)	59.47	15.65	37.42	37.90
Content + Ref (100cls)	<u>80.27</u>	<u>19.93</u>	<u>59.88</u>	<u>55.89</u>
Content (100cls)	48.59	8.28	28.66	22.15
Style ft.	25.98	3.96	12.95	10.29
Content (100cls)	<u>48.59</u>	<u>8.28</u>	<u>28.66</u>	<u>22.15</u>
Title+abstr.	15.48	2.14	7.38	6.15
Ref. contexts	21.47	2.73	9.20	7.39

Results on EMNLP dataset (article level)

Features	Acc	MAP	MAR	MRR
All features (top100cls)	<u>78.49</u>	<u>17.70</u>	<u>56.09</u>	<u>48.74</u>
All features (mid100cls)	51.68	11.09	39.49	33.91
All features (last100cls)	51.16	9.28	37.96	27.11
All features (262cls)	57.86	13.84	34.65	33.81
Content + Ref (100cls)	77.41	18.19	56.88	50.60
Content (100cls)	56.98	11.27	37.81	31.08
Style ft.	31.82	4.93	16.21	13.23
References	<u>78.49</u>	<u>18.79</u>	<u>58.04</u>	<u>52.26</u>
Content (100cls)	<u>56.98</u>	11.27	<u>37.81</u>	<u>31.08</u>
Title+abstr.	10.75	0.82	3.47	2.32
Ref. contexts	15.90	<u>16.83</u>	6.21	4.25

Conclusions:

- **Results show that references are the features that by far contribute the most to predicting the author(s) of an article.**
- ✓ **Feeding the extracted references into the deep network further boosts the predictive power of the references feature, reaching on its own an accuracy of 86.67% on ACL and 78.49% on EMNLP in an experiment looking at the top 100 most prolific authors in each dataset.**

- The more general text features (content level) are the second best predictor, whereas the style level features come last.
- With regard to the parts of the article text that seem to be most predictive, reference contexts seem to play a more important role than the title and abstract, even though using the full article content still gives the best results on both datasets.

- **As we go from the top 100 most prolific authors to the last (rare) authors, the performance keeps decreasing.**
- ✓ **For example, the accuracy on ACL at article level decreases from 87.88% (achieved for top 100 most prolific authors) to 50% (achieved for last 200 rare authors).**

- **It has been shown that the top most prolific authors of anonymized scientific articles can be predicted with high accuracy, with characteristics of the authors being apparent at all levels of the text: from content to style. Still, the most direct indicator of who the author of a paper might be comes from the papers that are referenced - both from the references themselves and from the citation context they occur in within the content of an article.**